RV Control System

Developer Setup Guide

Complete Environment Configuration for New Developers

Platform: React Native Mobile App + Raspberry Pi Server

Hardware: Raspberry Pi (CAN Bus) + Victron Cerbo GX (Modbus)

Protocol: RV-C (CAN 2.0B) over RV drop cable

Connectivity: WebSocket real-time sync

Table of Contents

1. What This Project Does
2. How the System Works (Architecture Overview)
3. What You Need Before Starting
4. Part A: Setting Up Your Development Computer
5. Part B: Setting Up the Raspberry Pi
6. Part C: Connecting the Pi to the RV
7. Part D: Running the Mobile App
8. Project File Structure Explained
9. Key Concepts You Need to Understand
10. Common Commands Reference
11. Troubleshooting
12. Glossary

1. What This Project Does

This project is a mobile app that lets you control an RV (recreational vehicle) from your phone. Think of it like a smart home app, but for an RV. You can control:

- Lights – Turn on/off and dim 14+ individual lights (kitchen, bathroom, bedroom, porch, etc.)
- Climate – AC cooling, furnace heating, fan speeds, dehumidify mode, night mode
- Water Systems – Water pump and water heater on/off
- Awning – Extend, retract, and stop the exterior awning
- Fans – Bathroom fan and bay vent fan
- Bar Lift – Motorized bar lift up/down
- Power Monitoring – Live battery, solar, shore power, and AC load data from Victron system

The phone app talks to a small computer (Raspberry Pi) that is physically wired into the RV's internal network. The Pi translates your phone commands into the electrical signals the RV understands.

2. How the System Works (Architecture Overview)

The Big Picture

There are three main pieces that work together:

Component	What It Is	What It Does
Mobile App	React Native app on your phone	The user interface – buttons, sliders, status displays
Raspberry Pi Server	Small Linux computer in the RV	Translates app commands into CAN bus messages the RV understands
RV-C Network	The RV's built-in wiring (CAN bus)	Carries electrical signals to lights, AC, pumps, etc.

How a Command Flows

When you tap "Kitchen Lights" on the app, here is exactly what happens:

1. Your phone app sends an HTTP request to the Raspberry Pi at http://192.168.8.200:3000/api/execute with the command name "kitchen_lights_toggle".
2. The Pi server (server.js) looks up that command name and finds the matching CAN bus hex code(s) to send.
3. The Pi sends the hex command over its CAN interface (can0) onto the RV-C network through the drop cable.
4. The RV's lighting controller receives the CAN message and physically toggles the kitchen lights.
5. The Pi also broadcasts a WebSocket message so any other connected devices update their UI in real time.

Network Addresses

Device	IP Address	Port	Purpose
Raspberry Pi	192.168.8.200	3000	REST API + WebSocket server
Victron Cerbo GX	169.254.9.43	502	Modbus TCP for power data

> **NOTE:** The Pi connects to the Victron system via Ethernet using a link-local address (169.254.x.x). The Pi connects to the mobile app via WiFi on the 192.168.8.x network.

3. What You Need Before Starting

Hardware You Need

Item	Why You Need It
A computer (Mac, Windows, or Linux)	This is your development machine where you write code
A Raspberry Pi (3B+ or 4 recommended)	The server that lives in the RV
A microSD card (32GB or larger)	The Pi's "hard drive"
A CAN bus HAT or USB adapter for the Pi	Lets the Pi talk on the RV's CAN network (e.g., Waveshare RS485 CAN HAT or PCAN-USB)
An Ethernet cable	To connect the Pi to the Victron Cerbo GX
The RV's drop cable	Pre-existing cable in the RV that connects to the RV-C CAN bus network
A phone (iOS or Android)	To run the mobile app
A USB keyboard + monitor (temporary)	For initial Pi setup only – you can remove these later once SSH is configured

Software You Need to Install on Your Computer

Software	Where to Get It	What It Does
Node.js (v18 or newer)	https://nodejs.org	JavaScript runtime – powers both the Pi server and the app build tools
npm	(comes with Node.js)	Package manager for installing code libraries
Git	https://git-scm.com	Version control – for downloading and managing the project code
VS Code (recommended)	https://code.visualstudio.com	Code editor with great extensions for this project
React Native CLI	Installed via npm	Build tool for the mobile app
Expo CLI (if using Expo)	Installed via npm	Alternative React Native build tool
Raspberry Pi Imager	https://www.raspberrypi.com/software/	Tool to flash the operating system onto the Pi's SD card

Accounts You Might Need

Account	Why
GitHub account	To clone/push the project repository
Apple Developer account	Only if building for iOS (costs $99/year)
Google Play Console	Only if building for Android distribution

4. Part A: Setting Up Your Development Computer

4.1 – Install Node.js

Node.js is the engine that runs JavaScript outside of a web browser. Both the Raspberry Pi server and the mobile app build tools need it.

Step 1: Go to https://nodejs.org and download the LTS (Long Term Support) version.

Step 2: Run the installer. Accept all defaults.

Step 3: Open a terminal (Mac/Linux) or Command Prompt (Windows) and verify:

```
node --version
npm --version
```

You should see version numbers (e.g., v18.17.0 and 9.6.7). If you get "command not found," restart your terminal.

4.2 – Install Git

Step 1: Go to https://git-scm.com/downloads and install for your OS.

Step 2: Verify installation:

```
git --version
```

Step 3: Configure your identity (use your real name and email):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

4.3 – Clone the Project Repository

Step 1: Open your terminal and navigate to where you want the project:

```
cd ~/Documents
mkdir RVControl
cd RVControl
```

Step 2: Clone the repository (replace URL with the actual repo URL):

```
git clone https://github.com/YOUR-ORG/rv-control-app.git
cd rv-control-app
```

Step 3: Install the project dependencies:

```
npm install
```

This will download all the libraries the project needs. It may take a few minutes.

> **NOTE:** If you see errors about "peer dependencies," that's usually OK. Look for actual ERROR messages.

4.4 – Install React Native Development Tools

For Android Development:

Step 1: Download and install Android Studio from https://developer.android.com/studio

Step 2: During installation, make sure these are checked: Android SDK, Android SDK Platform, Android Virtual Device.

Step 3: Open Android Studio → Settings → SDK Manager → install Android 13 (API 33) or newer.

Step 4: Set up the ANDROID_HOME environment variable. Add to your shell profile (~/.bashrc, ~/.zshrc, or system environment variables):

```
# Mac/Linux - add to ~/.zshrc or ~/.bashrc
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/emulator
export PATH=$PATH:$ANDROID_HOME/platform-tools
# Windows - add to System Environment Variables
# ANDROID_HOME = C:\Users\YourName\AppData\Local\Android\Sdk
```

For iOS Development (Mac only):

Step 1: Install Xcode from the Mac App Store (it's free but large, ~12GB).

Step 2: Open Xcode once to accept the license agreement.

Step 3: Install CocoaPods (iOS dependency manager):

```
sudo gem install cocoapods
```

Step 4: Navigate to the iOS folder in the project and install pods:

```
cd ios
pod install
cd ..
```

4.5 – Install VS Code Extensions (Recommended)

Extension Name	What It Does
ES7+ React/Redux/React-Native snippets	Code shortcuts for React Native
Prettier	Auto-formats your code
ESLint	Catches common JavaScript errors
Remote - SSH	Lets you edit code on the Raspberry Pi directly from VS Code
Thunder Client or REST Client	Test API endpoints without leaving the editor

5. Part B: Setting Up the Raspberry Pi

The Raspberry Pi is the brain that sits inside the RV. It runs a Node.js server that listens for commands from the phone app and translates them into CAN bus messages.

5.1 – Flash the Operating System

Step 1: Download Raspberry Pi Imager from https://www.raspberrypi.com/software/

Step 2: Insert your microSD card into your computer.

Step 3: Open Raspberry Pi Imager and select:

- • OS: Raspberry Pi OS (64-bit) – the "Lite" version is fine (no desktop needed)
- • Storage: Your microSD card

Step 4: Click the gear icon (Advanced Options) and configure:

- • Set hostname: rvcontrol (or whatever you prefer)
- • Enable SSH: Use password authentication
- • Set username: pi
- • Set password: (choose a secure password and write it down)
- • Configure WiFi: Enter your RV's WiFi network name and password

Step 5: Click "Write" and wait for it to finish.

Step 6: Put the SD card in the Raspberry Pi and power it on.

> **WARNING:** Write down your Pi's username and password! You will need them every time you connect via SSH.

5.2 – Connect to the Pi via SSH

SSH lets you control the Pi from your computer's terminal, without needing a monitor and keyboard plugged into the Pi.

Step 1: Wait about 60 seconds after powering on the Pi for it to boot.

Step 2: From your computer's terminal, connect:

```
ssh pi@rvcontrol.local
```

Or if that doesn't work, use the Pi's IP address:

```
ssh pi@192.168.8.200
```

Step 3: Type "yes" when asked about the fingerprint, then enter your password.

> **TIP:** If you don't know the Pi's IP, you can check your router's admin page or use: nmap -sn 192.168.8.0/24

5.3 – Update the Pi and Install Core Software

Run these commands on the Pi (after connecting via SSH):

Step 1: Update the operating system:

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Node.js (the Pi needs this to run the server):

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

Step 3: Verify Node.js installed correctly:

```
node --version
npm --version
```

Step 4: Install CAN bus utilities (for communicating with the RV):

```
sudo apt install -y can-utils
```

Step 5: Install other useful tools:

```
sudo apt install -y git nano screen
```

5.4 – Configure the CAN Bus Interface

The CAN bus is the communication network inside the RV. The Pi needs a CAN interface (either a HAT or USB adapter) to send and receive messages on this network. The RV uses the RV-C protocol which runs at 250 kbps (250,000 bits per second).

Step 1: If using a CAN HAT (like Waveshare), enable SPI first:

```
sudo raspi-config
```

Navigate to: Interface Options → SPI → Enable. Then reboot: sudo reboot

Step 2: Add the CAN interface configuration. Edit the network config:

```
sudo nano /etc/network/interfaces.d/can0
```

Step 3: Add these lines to the file:

```
auto can0
iface can0 inet manual
    pre-up /sbin/ip link set can0 type can bitrate 250000
    up /sbin/ifconfig can0 up
    down /sbin/ifconfig can0 down
```

> **NOTE:** The bitrate 250000 is critical – this is the RV-C standard speed. Using the wrong speed means the Pi cannot communicate with the RV.

Step 4: If using a Waveshare CAN HAT, also add the overlay. Edit /boot/config.txt:

```
sudo nano /boot/config.txt
```

Add this line at the bottom:

```
dtoverlay=mcp2515-can0,oscillator=12000000,interrupt=25,spimaxfrequency=2000000
```

Step 5: Reboot the Pi:

```
sudo reboot
```

Step 6: After reboot, verify the CAN interface is working:

```
ip link show can0
```

You should see "can0" with state "UP". If it says "DOWN" or doesn't appear, double-check your HAT wiring and configuration.

Step 7: Test that you can see CAN traffic from the RV (the RV must be powered on):

```
candump can0
```

You should see lines of hex data scrolling by. Press Ctrl+C to stop. If you see nothing, check your drop cable connection.

5.5 – Configure the Static IP Address

The mobile app is hardcoded to reach the Pi at 192.168.8.200. You need to set a static IP so it doesn't change.

Step 1: Edit the DHCP configuration:

```
sudo nano /etc/dhcpcd.conf
```

Step 2: Add these lines at the bottom:

```
interface wlan0
static ip_address=192.168.8.200/24
static routers=192.168.8.1
static domain_name_servers=8.8.8.8 8.8.4.4
```

Step 3: Restart networking:

```
sudo systemctl restart dhcpcd
```

> **NOTE:** If your RV's WiFi router uses a different subnet (not 192.168.8.x), you'll need to update both this config AND the API_URL in the app code (API/rvAPI.js).

5.6 – Copy the Server Files to the Pi

Now you need to get the server code onto the Pi. The server files include server.js (the main server) and supporting files.

Step 1: On your development computer, navigate to the server folder in the project:

```
cd ~/Documents/RVControl/rv-control-app/server
```

Step 2: Copy the files to the Pi using SCP (Secure Copy):

```
scp -r ./* pi@192.168.8.200:~/rv-server/
```

This copies all server files to a folder called "rv-server" in the Pi's home directory.

Step 3: SSH into the Pi and install the server's dependencies:

```
ssh pi@192.168.8.200
cd ~/rv-server
npm install
```

Step 4: Test that the server starts:

```
node server.js
```

You should see output like "Server running on port 3000" and "CAN interface: can0". Press Ctrl+C to stop.

5.7 – Set Up the Server to Run Automatically

You want the server to start automatically when the Pi boots up, so you don't have to manually start it every time.

Step 1: Install PM2 (a process manager that keeps the server running):

```
sudo npm install -g pm2
```

Step 2: Start the server with PM2:

```
cd ~/rv-server
pm2 start server.js --name rv-control
```

Step 3: Tell PM2 to start on boot:

```
pm2 save
pm2 startup
```

PM2 will print a command starting with "sudo". Copy and paste that entire command and run it.

Step 4: Verify it's running:

```
pm2 status
```

You should see "rv-control" with status "online".

> **TIP:** Useful PM2 commands: "pm2 logs rv-control" shows server output. "pm2 restart rv-control" restarts the server. "pm2 stop rv-control" stops it.

6. Part C: Connecting the Pi to the RV

6.1 – The Drop Cable

The RV has an internal communication network called the RV-C bus (based on CAN bus technology). All the "smart" components in the RV – lights, AC, furnace, water heater – are connected to this network. The "drop cable" is a pre-wired cable in the RV that gives you access to this network.

The drop cable typically has two important wires:

Wire	Color (typical)	Connection
CAN High (CANH)	Yellow or White	Connects to CAN_H on your CAN adapter
CAN Low (CANL)	Green or Blue	Connects to CAN_L on your CAN adapter
Ground (GND)	Black or Bare	Connects to GND on your CAN adapter

> **WARNING:** Double check wire colors against your specific RV's documentation. Reversing CAN_H and CAN_L will prevent communication. Connecting to the wrong wires could damage equipment.

6.2 – The Victron Connection

The RV also has a Victron Energy system (Cerbo GX) that monitors batteries, solar panels, shore power, and AC loads. The Pi connects to this separately via an Ethernet cable using Modbus TCP protocol.

Step 1: Connect an Ethernet cable from the Pi's Ethernet port to the Victron Cerbo GX's Ethernet port.

Step 2: The Victron uses a link-local address (169.254.9.43). The Pi should auto-configure a compatible link-local address on its Ethernet interface.

Step 3: Test the connection:

```
ping 169.254.9.43
```

If you get replies, the connection is working.

6.3 – Verify Everything Works Together

Step 1: Make sure the RV is powered on (house batteries on, or connected to shore power).

Step 2: Make sure the Pi is running and connected to WiFi.

Step 3: On the Pi, check the CAN bus is receiving data:

```
candump can0 | head -20
```

Step 4: Check the server is running:

```
pm2 status
```

Step 5: From your phone or computer on the same WiFi, test the API:

```
# In a web browser or using curl:
curl http://192.168.8.200:3000/api/status
```

You should get a JSON response showing the server status and CAN interface state.

Step 6: Test a command (toggle kitchen lights):

```
curl -X POST http://192.168.8.200:3000/api/execute \
  -H "Content-Type: application/json" \
  -d '{"command": "kitchen_lights_toggle"}'
```

The kitchen lights in the RV should toggle on or off!

7. Part D: Running the Mobile App

7.1 – Make Sure Your Phone and Pi Are on the Same Network

The app communicates with the Pi over WiFi. Both your phone and the Pi must be connected to the same WiFi network (the RV's router, typically on the 192.168.8.x subnet).

7.2 – Start the Metro Bundler

The Metro bundler is a development tool that compiles and serves the React Native app to your phone.

Step 1: Open a terminal on your development computer.

Step 2: Navigate to the project root:

```
cd ~/Documents/RVControl/rv-control-app
```

Step 3: Start the bundler:

```
npx react-native start
```

Leave this terminal window running. It will show a menu and compilation progress.

7.3 – Run on Android

Step 1: Connect your Android phone via USB with USB debugging enabled. Or start an Android emulator from Android Studio.

Step 2: Open a new terminal (keep Metro running) and run:

```
npx react-native run-android
```

Step 3: The app should build, install on your phone, and open automatically.

7.4 – Run on iOS (Mac only)

Step 1: Make sure you installed pods (see Section 4.4).

Step 2: Open a new terminal and run:

```
npx react-native run-ios
```

Or open the .xcworkspace file in Xcode and click the Run button.

7.5 – Verify the App Connects to the Pi

When the app launches, it will try to connect to the Pi at http://192.168.8.200:3000. If successful, you'll see the RV control interface. If it shows connection errors:

- Confirm your phone is on the same WiFi as the Pi.
- Confirm the Pi server is running (pm2 status on the Pi).
- Try accessing http://192.168.8.200:3000/api/status in your phone's web browser.

- Check the IP address in API/rvAPI.js matches your Pi’s actual IP.

8. Project File Structure Explained

Here is what each major file and folder in the project does:

Mobile App Files (on your development computer)

File / Folder	Purpose
API/rvAPI.js	The core API client. Defines the Pi's address (192.168.8.200:3000) and all the functions for sending commands (toggle lights, set brightness, etc.)
API/RVControlServices.js	Higher-level service modules organized by category: AwningService, WaterService, ClimateService, LightService, FanService, BarLiftService
API/VictronAPI.js	API client for the Victron energy system. Talks to the Pi which proxies requests to the Cerbo GX at 169.254.9.43:502
API/VictronEnergyService.js	Manages Victron data with auto-polling, caching, and fallback simulation mode when the real API is unavailable
API/RVStateManager/	Central state management system with two files:
RVStateManager.js	Singleton that tracks all RV state (lights, climate, water, power), listens to the CAN bus via WebSocket, handles offline queuing, and syncs across multiple devices
RVStateHooks.js	React hooks (useRVState, useRVLights, useRVClimate, useRVWater) that let UI components subscribe to live state changes
Service/CANBusListener.js	Creates and manages a WebSocket connection to listen for raw CAN bus messages from the Pi in real time

Raspberry Pi Server Files (on the Pi)

File	Purpose
server.js	The main Express.js server. Defines all API routes (/api/execute, /api/raw, /api/brightness, etc.), stores the CAN command lookup table, and manages the WebSocket server
package.json	Lists the server's npm dependencies (express, ws, socketcan or similar)

Key Relationships Between Files

The app code and server code connect through HTTP and WebSocket. The rvAPI.js file is the single point of connection – every command from the app flows through it. The RVControlServices.js file provides a friendlier interface on top of rvAPI.js, organized by RV system (lights, climate, water, etc.). The RVStateManager listens to WebSocket broadcasts from the server to keep the UI in sync with the actual RV state.

9. Key Concepts You Need to Understand

9.1 – CAN Bus & RV-C Protocol

CAN (Controller Area Network) is a communication standard used in vehicles. RV-C is a protocol built on top of CAN specifically for RVs. Every command is a hexadecimal message with a specific format.

A typical command looks like: 19FEDB9F#09FFC8012D00FFFF

Breaking that down:

Part	Example	Meaning
CAN ID	19FEDB9F	Identifies the type of message (FEDB = DC_DIMMER_COMMAND_2)
Separator	#	Separates the ID from the data
Data Byte 1	09	Instance number (which specific device, e.g., 09 = awning extend motor)
Data Bytes 2-8	FFC8012D00FFFF	Command parameters (brightness, duration, command type, etc.)

9.2 – Instance Numbers

Each physical device on the CAN bus has a unique "instance" number. Here are the ones defined in this project:

Instance (Decimal)	Instance (Hex)	Device
21	0x15	Bathroom Light
22	0x16	Vibe Light
23	0x17	Vanity Light
24	0x18	Dinette Lights
25	0x19	Awning Lights
26	0x1A	Kitchen Lights
27	0x1B	Bedroom Overhead Light
28	0x1C	Shower Lights
29	0x1D	Under Cabinet Lights
30	0x1E	Hitch Lights
31	0x1F	Porch Lights
32	0x20	Strip Lights

34	0x22	Left Reading Lights
35	0x23	Right Reading Lights
43	0x2B	Water Heater
44	0x2C	Water Pump

9.3 – WebSocket Real-Time Sync

The server runs a WebSocket server (at ws://192.168.8.200:3000) alongside the REST API. When any device sends a command, the server broadcasts it to all connected clients via WebSocket. This is how multiple phones can control the RV and all stay in sync – when one phone toggles a light, all other connected phones see the change instantly.

9.4 – Victron Energy Monitoring

The Victron Cerbo GX is a separate system that monitors the RV's power. The Pi proxies data from the Cerbo GX to the app via the /api/victron/* endpoints. The VictronEnergyService in the app polls this data every 5 seconds and includes a built-in simulation mode that activates automatically when the real Victron system is unreachable. This is useful for development and testing when you're not connected to the RV.

9.5 – The State Manager Pattern

The RVStateManager is a singleton (only one instance exists) that serves as the "single source of truth" for all RV state. It tracks whether each light is on/off, brightness levels, climate settings, and more. UI components subscribe to it using custom React hooks (useRVLights, useRVClimate, etc.) and automatically re-render when relevant state changes.

10. Common Commands Reference

Development Computer Commands

Command	What It Does
npm install	Installs all project dependencies
npx react-native start	Starts the Metro bundler (dev server for the app)
npx react-native run-android	Builds and runs the app on Android
npx react-native run-ios	Builds and runs the app on iOS (Mac only)
npm test	Runs the test suite
git pull	Downloads latest code changes from the repository
git status	Shows which files you've changed
git add . && git commit -m "message"	Saves your changes locally
git push	Uploads your saved changes to the repository

Raspberry Pi Commands

Command	What It Does
ssh pi@192.168.8.200	Connect to the Pi remotely
pm2 status	Check if the server is running
pm2 logs rv-control	View real-time server logs
pm2 restart rv-control	Restart the server (after code changes)
pm2 stop rv-control	Stop the server
candump can0	View raw CAN bus traffic (Ctrl+C to stop)
cansend can0 19FEDB9F#09FFC8012D00FFFF	Manually send a CAN command (useful for testing)
ip link show can0	Check CAN interface status
sudo ip link set can0 up type can bitrate 250000	Manually bring up CAN interface
ifconfig	Show all network interfaces and IP addresses
ping 169.254.9.43	Test Victron Cerbo GX connection

Testing API Endpoints from Your Computer

Endpoint	Method	Description

http://192.168.8.200:3000/api/status	GET	Server status and CAN interface info
http://192.168.8.200:3000/api/commands	GET	List all available commands
http://192.168.8.200:3000/api/execute	POST	Execute a named command (body: {"command": "name"})
http://192.168.8.200:3000/api/raw	POST	Send a raw CAN hex command (body: {"canCommand": "hex"})
http://192.168.8.200:3000/api/brightness	POST	Set light brightness (body: {"lightId": "id", "level": 50})
http://192.168.8.200:3000/api/brightness-status	GET	Get current brightness of all lights
http://192.168.8.200:3000/api/victron/all	GET	Get all Victron power data
http://192.168.8.200:3000/api/victron/battery	GET	Get battery status only

11. Troubleshooting

Problem	Solutions
"Cannot connect to server" in the app	1) Check your phone is on the same WiFi as the Pi. 2) Run "pm2 status" on the Pi to confirm the server is running. 3) Try http://192.168.8.200:3000/api/status in a browser. 4) Check the IP in API/rvAPI.js matches your Pi's actual IP.
"can0: No such device" on the Pi	1) Check the CAN HAT is properly seated on the GPIO pins. 2) Verify /boot/config.txt has the correct dtoverlay line. 3) Run: sudo ip link set can0 up type can bitrate 250000 4) Reboot the Pi: sudo reboot
candump shows no data	1) Make sure the RV is powered on (house batteries active). 2) Check the drop cable connections – CAN_H, CAN_L, and GND must be correct. 3) Try reversing CAN_H and CAN_L if you're unsure of the wiring. 4) Check for a termination resistor – the CAN bus may need a 120Ω resistor.
Lights don't respond to commands	1) Verify the server is running and the API returns success. 2) Check candump – you should see outgoing messages when you send commands. 3) Make sure the RV's house batteries are on (12V systems need power to respond). 4) Try sending the command directly: cansend can0 19FEDB9F#1AFF00050000FFFF
Victron data shows "simulation"	1) This is normal when the Pi isn't connected to the Cerbo GX via Ethernet. 2) Check the Ethernet cable between Pi and Cerbo GX. 3) Run: ping 169.254.9.43 from the Pi. 4) Make sure Modbus TCP is enabled in the Cerbo GX settings.
"npm install" fails	1) Check your internet connection. 2) Try deleting node_modules and package-lock.json, then run npm install again. 3) If on the Pi, make sure you have enough disk space: df -h 4) Try: npm cache clean --force
Metro bundler crashes or freezes	1) Stop it (Ctrl+C) and restart: npx react-native start --reset-cache

	2) Delete temporary files: rm -rf node_modules/.cache 3) On Mac, also try: watchman watch-del-all
App builds but shows a blank white screen	1) Check the Metro bundler terminal for error messages (red text). 2) On Android, shake the phone to open the dev menu → Reload. 3) Make sure the phone and computer are on the same network. 4) Check that the bundler URL is correct in the dev menu settings.

12. Glossary

Term	Definition
API	Application Programming Interface – a way for software to communicate with other software. In this project, the Pi runs a REST API that the phone app talks to.
CAN Bus	Controller Area Network – a robust communication standard used in vehicles. Devices send and receive short hexadecimal messages on shared wires.
RV-C	A protocol (set of rules) built on top of CAN bus specifically designed for RVs. Defines standard message formats for lights, HVAC, tanks, etc.
Drop Cable	A pre-installed cable in the RV that provides access to the RV-C CAN bus network.
DGN	Data Group Number – identifies the type of RV-C message (e.g., FEDB = DC Dimmer Command).
Instance	A unique number identifying a specific device on the CAN bus (e.g., Instance 26 = Kitchen Lights).
Hex / Hexadecimal	Base-16 numbering system using digits 0-9 and letters A-F. CAN messages are written in hex.
Node.js	A JavaScript runtime that lets you run JavaScript outside a web browser – used for the Pi server.
npm	Node Package Manager – a tool for installing JavaScript libraries and managing dependencies.
React Native	A framework for building mobile apps using JavaScript and React. Write once, runs on both iOS and Android.
SSH	Secure Shell – a way to remotely log into another computer over a network and run commands.
SCP	Secure Copy – a way to copy files between computers over SSH.
WebSocket	A protocol for real-time two-way communication between a client and server. Used for live state sync.
REST	Representational State Transfer – a style of API using standard HTTP methods (GET, POST, etc.).
PM2	A process manager for Node.js that keeps your server running and restarts it if it crashes.
Modbus TCP	A communication protocol used by the Victron Cerbo GX for reading power system data.
Metro Bundler	The development server used by React Native to compile and serve the app to your phone during development.

Cerbo GX	A Victron Energy device that monitors and manages the RV's power system (battery, solar, inverter, etc.).
HAT	Hardware Attached on Top – an add-on board that plugs onto a Raspberry Pi's GPIO pins.
GPIO	General Purpose Input/Output – the row of metal pins on the Raspberry Pi used to connect hardware.
Bitrate	The speed of communication on the CAN bus, measured in bits per second. RV-C uses 250,000 bps.
Singleton	A design pattern where only one instance of a class exists. The RVStateManager is a singleton.